

Zelig Styler SDK — iOS Integration Guide

This guide covers embedding the Zelig Styler widget into a third-party iOS app. Shoppers build looks, try on outfits, and add items to cart; your app handles dismissal and cart operations through a small delegate / closure bridge.

The SDK ships as a **pre-built closed-source binary** (`ZeligStylerSDK.xcframework`) — no source code is included.

Prerequisites

- iOS 13.0+
- Xcode 14+ (Xcode 15+ requires one extra build setting for CocoaPods — see Option B, step 4)
- Swift 5.7+
- Your **storeId** and **storeKey** — provided by your Zelig solutions engineer

Release artifacts

Current version: 1.0.1 — the URLs below are pinned to it. When Zelig publishes a newer native SDK, bump these version strings (see Step 10).

Artifact	URL
XCFramework zip (Option A)	https://storage.googleapis.com/ios-widget-sdk-zelig-prod/ZeligStylerSDK-1.0.1.xcframework.zip
CocoaPods podspec (Option B)	https://raw.githubusercontent.com/wearezelig/ZeligStylerSDK-binary/1.0.1/ZeligStylerSDK.podspec
SPM binary package (Option C)	repo https://github.com/wearezelig/ZeligStylerSDK-binary.git — pin by version/tag (1.0.1)

All three channels download the **same** prod XCFramework from GCS. Prefer pinning a **tag / version** (shown above), not **main**, so your app store build stays reproducible.

Step 1 — Install the SDK (pick one)

All three options deliver the same pre-built binary.

Use only one at a time. Before switching, fully remove the previous integration (dragged-in framework, pod entries, or SPM package), otherwise you'll see **Missing package product** or duplicate-module errors.

Option A — Direct XCFramework (simplest, no tooling required)

1. Download and unzip:

```
https://storage.googleapis.com/ios-widget-sdk-zelig-prod/ZeligStylerSDK-1.0.1.xcframework.zip
```

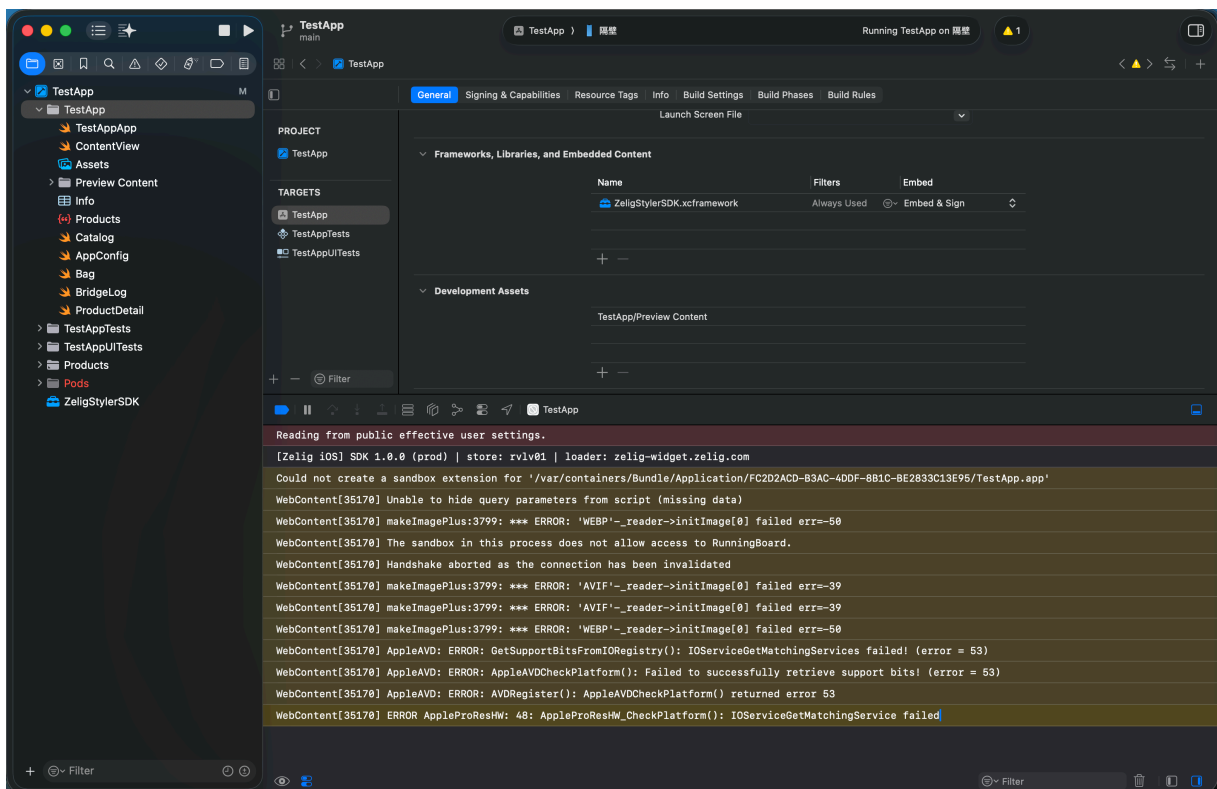
2. Drag **ZeligStylerSDK.xcframework** into your Xcode project. In the dialog that appears, make sure your **app target** is checked. Either "Reference files in place" or "Copy" works.

3. In your target's **General → Frameworks, Libraries, and Embedded Content**, set **ZeligStylerSDK.xcframework** to **Embed & Sign**. (Xcode may default to "Do Not Embed" — you must change it.)

That's it — no package manager involved.

This option has no automatic version management or checksum verification. Upgrading means downloading the **new versioned zip URL** and swapping the file. For SHA-256-verified pinned installs, prefer Option B or C.

Upgrading (example → 1.0.1): download **.../ZeligStylerSDK-1.0.1.xcframework.zip** → remove the old **ZeligStylerSDK.xcframework** from the project navigator → drag in the new one → confirm Embed & Sign → rebuild and ship.



Option B — CocoaPods

1. Add one line to your **Podfile**:

```
platform :ios, '13.0'
use_frameworks!

target 'YourApp' do
  # Pin a tagged release (recommended – keep builds
  reproducible):
  pod 'ZeligStylerSDK', :podspec =>
    'https://raw.githubusercontent.com/wearezelig/ZeligStylerSDK-
    binary/1.0.1/ZeligStylerSDK.podspec'
end
```

CocoaPods fetches the podspec from the binary repo tag (GitHub raw); the podspec downloads the pre-built zip from GCS and verifies SHA-256.

2. Run:

```
pod install
```

A successful install prints **Installing ZeligStylerSDK (1.0.1)**.

3. **Always open `YourApp.xcworkspace` from this point on, not `.xcodproj`**. Opening the wrong file causes "no such module 'ZeligStylerSDK'" at compile time.
4. **Required on Xcode 15+:** target → Build Settings → search **User Script Sandboxing** → set to **No** (both Debug and Release). Skipping this causes **Command PhaseScriptExecution failed with a nonzero exit code** — Xcode 15's sandbox blocks CocoaPods' framework-embed script on every pod-based project.

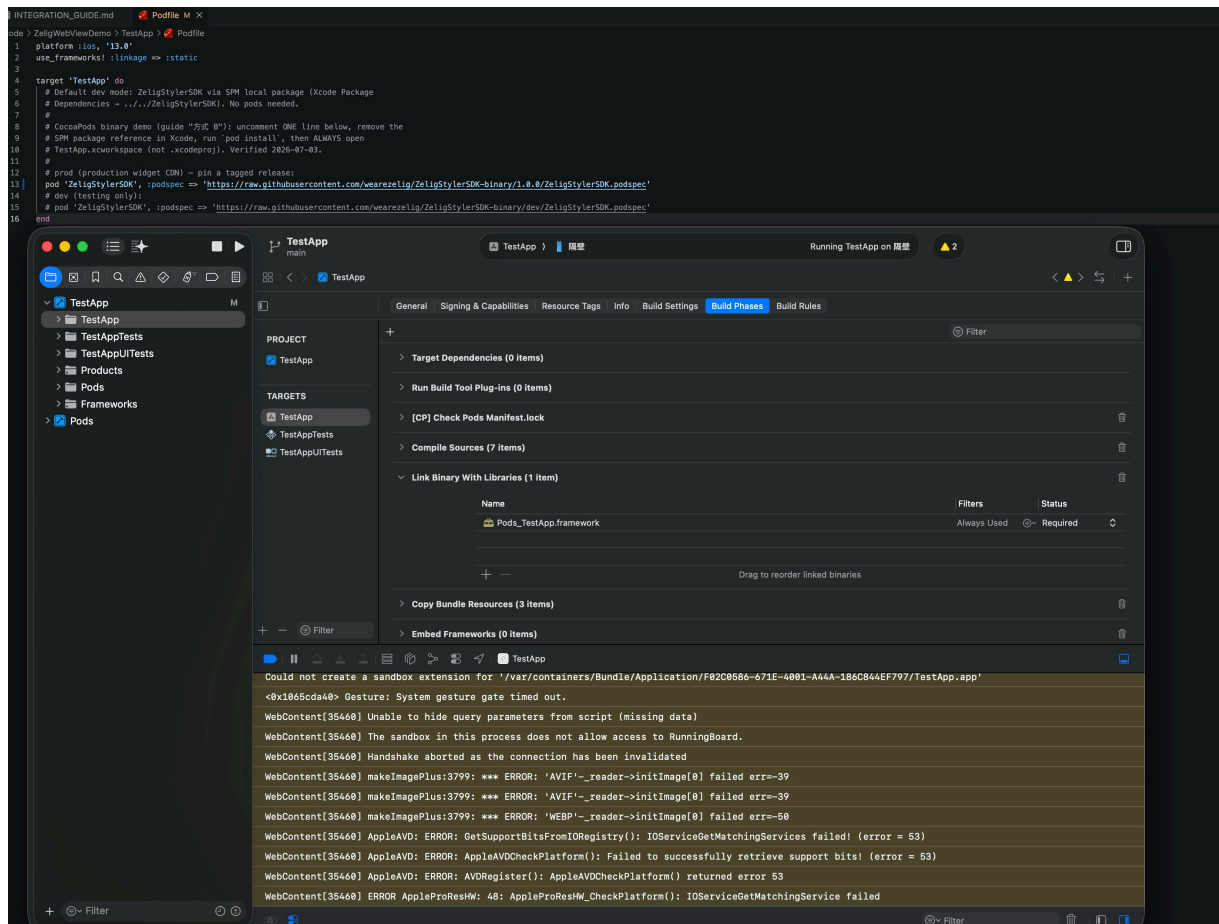
Upgrading (example → 1.0.1): change the tag in the Podfile URL, then reinstall:

```
pod 'ZeligStylerSDK', :podspec =>
  'https://raw.githubusercontent.com/wearezelig/ZeligStylerSDK-
  binary/1.0.1/ZeligStylerSDK.podspec'
```

```
rm -rf Pods Podfile.lock
pod install
```

Installing ZeligStylerSDK (1.0.1) in the output confirms success. Rebuild and ship.

Changing only **Podfile.lock** / re-running **pod install** **without** bumping the tag URL will keep you on **1.0.1**.



Option C — Swift Package Manager

The SDK ships as a binary Swift package. Pin it by **version**, not branch.

1. Xcode: **File** → **Add Package Dependencies...**
2. Paste the **git repo URL** — not the zip URL. (Pasting a zip URL prompts for a username/password; Xcode downloads and SHA-256-verifies the zip automatically based on the package's declaration.)

```
https://github.com/wearezelig/ZeligStylerSDK-binary.git
```

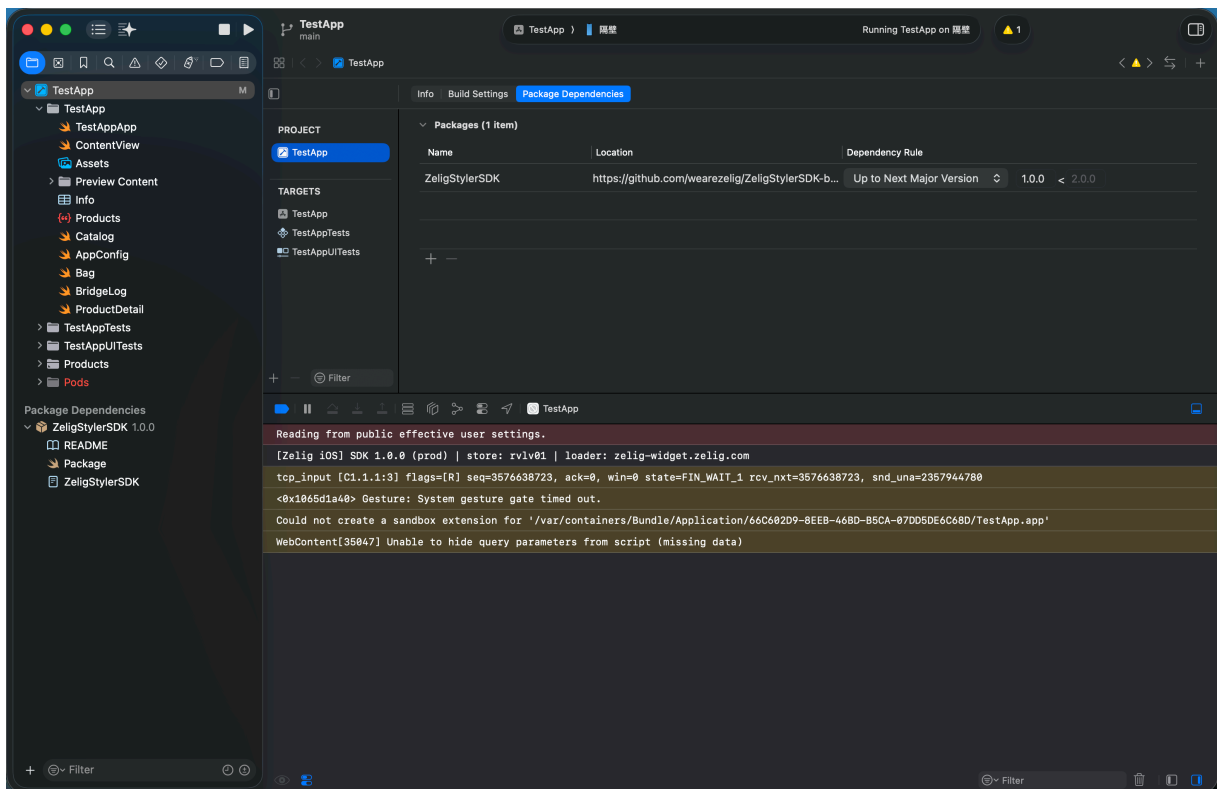
3. Set the **Dependency Rule** to **Up to Next Major Version**, starting at **1.0.1**.
That locks you to the **1.x** line (safe under semver) without picking up a breaking **1.0.1** automatically.
Use **Exact Version 1.0.1** only if your release process requires freezing a single binary until an explicit bump.
4. **Add to Project** → select your app project → **Add Package**. In the sheet that appears, **Add to Target** → select your app target → **Add**.

5. Verify: the Project Navigator shows **zeligstylersdk-binary**, and your target's **General → Frameworks, Libraries, and Embedded Content** lists **ZeligStylerSDK**.

Upgrading within 1.x (recommended rule = Up to Next Major):

1. Zelig tags a new release on **ZeligStylerSDK-binary** (e.g. **1.0.1**) and publishes a new GCS zip.
2. Xcode → **File** → **Packages** → **Update to Latest Package Versions** (or **Resolve Package Versions**). If Xcode won't refresh: **File** → **Packages** → **Reset Package Caches**, then update again.
3. Confirm the resolved version in the Project navigator / **Package.resolved** is the new tag.
4. Rebuild and ship.

Upgrading if you used Exact Version: change the rule to the new tag (e.g. **1.0.1**), resolve packages, rebuild, and ship.



Step 2 — Import

```
import ZeligStylerSDK
```

If this compiles, the installation succeeded. If you see **Unable to resolve module**: Option A — check target membership and the Embed setting; Option B — make sure you opened the **.xcworkspace**.

Step 3 — Store credentials

Your Zelig solutions engineer will provide your **storeId** and **storeKey**.

Security: treat **storeKey** like a password. Do not commit it to source control or embed it as a string literal in your binary. Load it from a secrets manager, a remote config endpoint, or your app's secure configuration layer.

Code examples in this guide use a placeholder store; substitute your own credentials.

Step 4 — Basic integration

Integration Note: The examples below show adding ZeligStyler to a product page, but you can integrate it anywhere in your app — product detail pages, cart, home recommendations, or a dedicated "Stylist" entry point. Replace class/view names with your actual implementation.

SwiftUI (recommended)

```
import SwiftUI
import ZeligStylerSDK

// Example: Add a "Build a Look" button to your existing view
struct ProductDetailView: View {
    let productCode: String // The product SKU/code to open on –
    // passed from your product page
    let userId: String?      // Current logged-in user ID from
    // your auth system, or nil for guests
    @State private var showStyler = false

    var body: some View {
        Button("Build a Look") { showStyler = true }
            .sheet(isPresented: $showStyler) {
                ZeligStylerView(
                    config: ZeligStylerConfig(
                        storeId: "YOUR_STORE_ID", //
                        // Replace with your storeId from Zelig
                        storeKey: "YOUR_STORE_KEY", //
                        // Replace with your storeKey from Zelig
                        productCode: productCode,
                        userId: userId
                    ),
                    onAddToCart: handleAddToCart, // Revolve's
                    // method (see Step 5)
                    onClose: { showStyler = false },
                    onError: { error in
                        print("Zelig error: \(error)")
                    }
                )
            }
    }
}
```

```

        .ignoresSafeArea()
    }
}

// Revolve's own method – the SDK calls it through the
`onAddToCart` hook so
// Revolve can add the item to its cart and run any logging /
analytics it needs.
// Revolve owns the body; full version in Step 5.
private func handleAddToCart(item: ZeligCartItem,
                             completion: @escaping (Bool) ->
Void) { /* see Step 5 */ }
}

```

UIKit

```

import UIKit
import ZeligStylerSDK

// Example: Add a "Build a Look" button to your existing view
controller
final class ProductViewController: UIViewController,
ZeligStylerDelegate {
    private var styler: ZeligStyler?

    func showStyler(productCode: String, userId: String? = nil) {
        let config = ZeligStylerConfig(
            storeId: "YOUR_STORE_ID",           // Replace with your
storeId from Zelig
            storeKey: "YOUR_STORE_KEY",        // Replace with your
storeKey from Zelig
            productCode: productCode,
            userId: userId                      // Pass logged-in
user ID, or nil for guests
        )
        let styler = ZeligStyler(config: config, delegate: self)
        self.styler = styler                  // retain the reference
        styler.present(from: self)
    }

    // Required delegate method – the SDK calls this when the
shopper taps Add to
    // Bag, so Revolve can add the item to its cart and run any
logging / analytics
    // it needs. Revolve owns the body; full version in Step 5.
    func zeligStyler(_ styler: ZeligStyler,
                     didRequestAddToCart item: ZeligCartItem,
                     completion: @escaping (Bool) -> Void) {

```

```

        // ... your cart call, then completion(true) /
        completion(false) – see Step 5
    }

    // Optional – defaults to dismissing the styler.
    func zeligStylerDidRequestClose(_ styler: ZeligStyler) {
        styler.dismiss(animated: true)
    }

    // Optional – called on load or initialization failures.
    func zeligStyler(_ styler: ZeligStyler, didFailWith error:
        ZeligStylerError) {
        print("Zelig error: \(error)")
    }
}

```

Preheating

The first open requires a full load (a few seconds). Call `prepare()` to preheat the widget asynchronously in the background — e.g. when the home screen appears; once preheating completes, the user can open Build a Look with no wait.

```

// SwiftUI – warm when the home view appears, and re-warm if the
// user changes
// Full reference:
ZeligWebViewDemo/TestApp/TestApp/ContentView.swift
.task(id: userId) {
    ZeligStyler(config: ZeligStylerConfig(
        storeId: "YOUR_STORE_ID",
        storeKey: "YOUR_STORE_KEY",
        productCode: "", // any value – the shared widget
        is warmed regardless
        userId: userId
    )).prepare()
}

```

```

// UIKit – warm ahead of the open (e.g. on viewDidLoad)
let styler = ZeligStyler(config: ZeligStylerConfig(
    storeId: "YOUR_STORE_ID",
    storeKey: "YOUR_STORE_KEY",
    productCode: "",
    userId: userId
), delegate: self)
styler.prepare()

```



```
// ... later, when the user opens:  
styler.present(from: self)
```

`prepare()` is scoped to the identity (`storeId + userId`): calling it again for the **same** identity is a no-op, but a **new** `userId` re-injects the cookie and reloads. So when `userId` can change at runtime (login, logout, account switch), preheat again under the new identity — otherwise the preheat done under the old identity no longer applies and the first open after a switch pays the full load again. In SwiftUI, keying the `.task` on `userId` (as above) does this automatically; in UIKit, call `prepare()` again with the new `userId` after the identity changes.

Step 5 — The add-to-cart bridge

When a shopper taps **Add to Bag** in the widget, the SDK calls `handleAddToCart`, a function **Revolve provides**, to add the item to the cart — the same role `window.addToBagWithData` plays in the web integration.

The example below shows `handleAddToCart` in full (pseudocode), including how `fromZelig` drives add-source attribution. `revolveCustomCartService` and `revolveCustomAnalyticsService` are pseudocode placeholders for Revolve's existing cart and tracking modules; the `insertPageTrackWithData` method is named after the web integration's own call, so the two sides line up.

SwiftUI — the `handleAddToCart` method you passed to `onAddToCart` in Step 4:

```
private func handleAddToCart(item: ZeligCartItem,  
                             completion: @escaping (Bool) -> Void)  
{  
    // `revolveCustomCartService` is Revolve's own cart layer –  
    not part of the SDK.  
    revolveCustomCartService.add(  
        sku: item.sku,  
        size: item.size,  
        quantity: item.quantity ?? 1,  
        fromZelig: item.fromZelig ?? true  
    ) { didSucceed in  
        // After the add, report the source with Revolve's own  
        tracking. The  
        // name/fields here just mirror web for easy comparison –  
        what matters  
        // is passing `fromZelig` through as the source flag.  
        revolveCustomAnalyticsService.insertPageTrackWithData(  
            addToCartFlag: true,  
            productCode: item.sku,  
            isZeligAddToBag: item.fromZelig ?? true    // web's  
            fromZelig  
        )  
    }  
    completion(didSucceed)  
}
```

```

        completion(didSucceed)    // true = added · false = failed
    }
}

```

Reference (SwiftUI): a runnable version ships in the sample app — `handleStylerAddToCart` in `ZeligWebViewDemo/TestApp/TestApp/ProductDetail.swift:206`. It's a SwiftUI implementation, so it maps to the SwiftUI code above (not the UIKit delegate below).

Real-world example (Revolve's production app). The same `onAddToCart` hook wired into a real cart layer. `RShopManager` / `RShoppingBag` are Revolve's own types — not part of the SDK — so this does not compile in the sample as-is (it's kept there as a comment); it shows the real shape:

```

private func handleAddToCart(_ item: ZeligCartItem, completion:
@escaping (Bool) -> Void) {
    guard let shopManager = RShopManager.sharedInstance() else {
        completion(false)
        return
    }

    let quantity = String(item.quantity ?? 1)

    shopManager.updateShoppingCart(
        forEmail: "",
        action: "add",
        code: item.sku,
        numSku: "1",
        prevSize: nil,
        qty: quantity,
        newSize: item.size,
        newCode: nil,
        itemDepartment: "",
        department: nil,
        pageType: "PDP",
        additionalQueryForURL: nil,
        success: { responseObject in
            DispatchQueue.main.async {
                guard let shoppingBag = responseObject as?
RShoppingBag else {
                    completion(false)
                    return
                }

                if shoppingBag.success {
                    shopManager.bagBadgeCount =
shoppingBag.numItems
                    NotificationCenter.default.post(name:

```

```

        NSNotification.Name.bagBadgeUpdated, object: nil)
            completion(true)
        } else {
            completion(false)
        }
    }
},
failure: { _ in
    DispatchQueue.main.async {
        completion(false)
    }
}
)
}

```

UIKit — the `ZeligStylerDelegate` method (the empty one from Step 4, now filled in).

As above, `revolveCustomCartService`, `revolveCustomAnalyticsService`, and `insertPageTrackWithData` are pseudocode placeholders — **replace them with the real definitions in your host app.**

```

func zeligStyler(_ styler: ZeligStyler,
                 didRequestAddToCart item: ZeligCartItem,
                 completion: @escaping (Bool) -> Void) {
    // `revolveCustomCartService` is Revolve's own cart layer –
    not part of the SDK.
    revolveCustomCartService.add(
        sku: item.sku,
        size: item.size,
        quantity: item.quantity ?? 1,
        fromZelig: item.fromZelig ?? true
    ) { didSucceed in
        // After the add, report the source with Revolve's own
        tracking. The
        // name/fields here just mirror web for easy comparison –
        what matters
        // is passing `fromZelig` through as the source flag.
        revolveCustomAnalyticsService.insertPageTrackWithData(
            addToCartFlag: true,
            productCode: item.sku,
            isZeligAddToBag: item.fromZelig ?? true // web's
            fromZelig
        )
        completion(didSucceed) // true = added · false = failed
    }
}

```

What **fromZelig** is for — add-source attribution. **insertPageTrackWithData** uses it to decide whether to tag the call as Zelig-originated, exactly as on web. The whole Zelig-specific difference is one line inside it:

```
// web (pseudocode): inside insertPageTrackWithData
if (isZeligAddToBag) u += "&navsrc=zeliglp"; // tag the source
only when it came from Zelig
```

If your cart API reports success differently — say it returns a **Result** or throws — map that to the boolean: **completion(true)** on the success path, **completion(false)** on the failure path.

The **item** carries these fields:

Field	Description
sku	Normalized product SKU (from the widget's code / productCode).
size	Selected size string (" all " for one-size items).
quantity	Quantity if the widget supplied one; otherwise nil .
fromZelig	true when the widget flagged this as a Zelig-originated add-to-cart; nil if absent.

The contract: you only need to call **completion** once — **true** if the item made it into the cart, **false** if not. The widget drives its in-modal UI from that signal and falls back to a ~2–4 s timeout; if the timeout fires first, its displayed state can drift from the cart, so respond promptly.

Step 6 — Configuration reference

Parameter	Type	Required	Default	Description
storeId	String	Yes	—	Your Zelig store identifier.
storeKey	String	Yes	—	Your store API secret, paired with storeId .
productCode	String	Yes for .productDetail	—	Hero SKU to open on. Pass "" for .buildALook / .myCloset .
userId	String?	No	nil	Your app's logged-in user ID. See Step 7.

Parameter	Type	Required	Default	Description
<code>entry</code>	<code>ZeligStylerEntry</code>	No	<code>.productDetail</code>	Which experience to open. See "Entry modes" below.

Entry modes

```
public enum ZeligStylerEntry { case productDetail, buildALook, myCloset }
```

Two ways the styler is launched:

CTA launch — `.productDetail` (default)

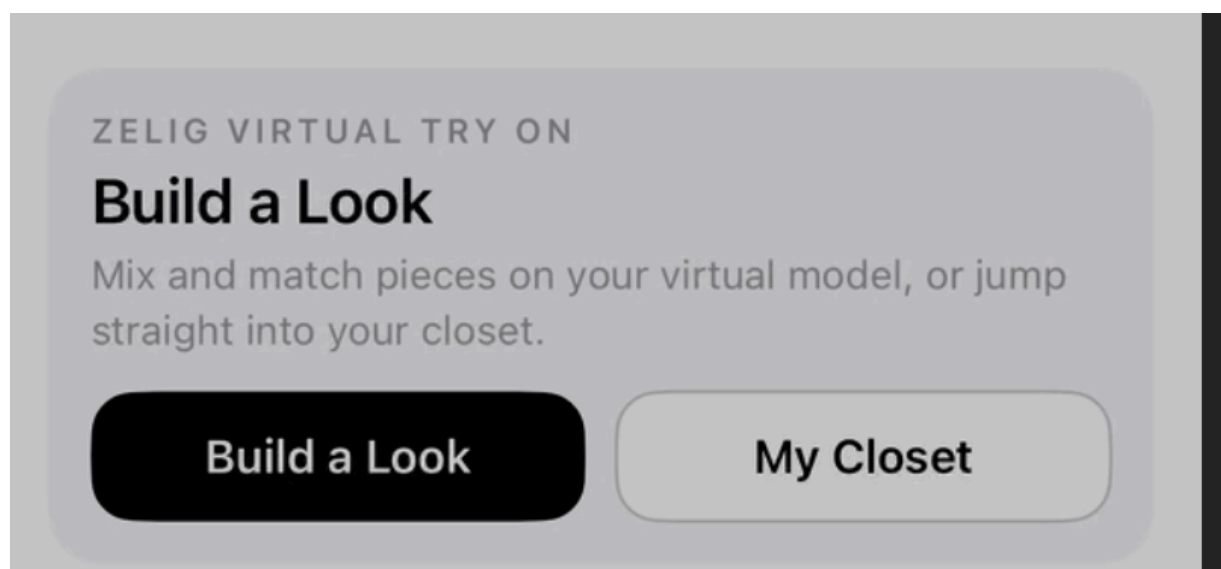
Opened from a product's Style-It CTA, on a specific product. `productCode` must be non-empty.

Banner launch — `.buildALook` / `.myCloset`

Opened from a banner entry point with no host-provided product (pass `productCode: ""`):

- `.buildALook` — opens the Build a Look experience. The widget picks the hero product itself (the shopper's last look, else the store's default recommendation).
- `.myCloset` — same as `.buildALook`, but lands directly on the My Closet tab.

SwiftUI landing: see `ZeligWebViewDemo/TestApp/TestApp/ContentView.swift` — `editorialBanner` renders the block and `.sheet(item:)` presents the styler.



```
// Open on a specific product (e.g. from a PDP):
ZeligStylerConfig(
    storeId: "YOUR_STORE_ID",
    storeKey: "YOUR_STORE_KEY",
```

```

        productCode: "PRODUCT_123"
    )

    // Entry points with no specific product:
    ZeligStylerConfig(
        storeId: "YOUR_STORE_ID",
        storeKey: "YOUR_STORE_KEY",
        productCode: "",
        entry: .buildALook
    )

    ZeligStylerConfig(
        storeId: "YOUR_STORE_ID",
        storeKey: "YOUR_STORE_KEY",
        productCode: "",
        userId: "USER_123", // Pass user ID for authenticated users
        entry: .myCloset
    )

```

Step 7 — Shopper identity & closet

`userId` tells the Zelig backend how to scope the shopper's closet:

- **Non-nil (signed in)** — the closet is tied to this user and syncs across devices. An anonymous closet is automatically inherited on first sign-in.
- **nil (guest)** — a device-local anonymous closet is used instead.

```

let config = ZeligStylerConfig(
    storeId: "YOUR_STORE_ID", // Replace with your storeId
    from Zelig
    storeKey: "YOUR_STORE_KEY", // Replace with your storeKey
    from Zelig
    productCode: productCode,
    userId: userId // Pass your logged-in user ID,
    or nil for guests
)

```

Step 8 — Error handling

When the widget fails to load or initialize, the SDK hands you a `ZeligStylerError` through the optional error callback — the `onError` closure in SwiftUI, the `zeligStyler(_:didFailWith:)` method in UIKit (both wired up in Step 4). Implementing it is optional but recommended, so you can show the shopper a graceful fallback instead of a stuck screen.

All cases occur during load / initialization — they do not interrupt an already-open session:

Case	What it means
<code>.loaderLoadFailed(URL)</code>	The widget loader couldn't be fetched from the CDN (the failing URL is attached).
<code>.widgetInitTimeout</code>	The widget didn't finish initializing within the timeout.
<code>.modalMountTimeout</code>	The widget's modal UI didn't mount in time.
<code>.webViewNavigationFailed(underlying:)</code>	The underlying <code>WKWebView</code> navigation failed (the underlying <code>Error</code> is attached).
<code>.hostPageUnavailable</code>	The page hosting the widget couldn't be reached.

This error callback is separate from the add-to-cart `completion` contract (which must fire on both success and failure) — see Step 5 for that.

Step 9 — Testing & debugging

Use the test SKUs your Zelig solutions engineer provides for your store.

Your main signal for diagnosing problems is the **error callback** (SwiftUI `onError` / UIKit `didFailWith`): handle the `ZeligStylerError` cases there, and add your own logging in that callback and in `onAddToCart` to trace behaviour.

Step 10 — SDK versions & updates (how Revolve / hosts upgrade)

Two layers update independently:

Layer	What it is	How it updates
Widget (JS)	Styler UI, looks, algorithms	Loaded at runtime from Zelig's CDN. No app store release required — shoppers get the latest widget the next time they open the styler.
Native SDK	<code>ZeligStylerSDK.xcframework</code> bridge	Version-pinned in your Xcode project. You only bump it when Zelig ships a new native release (bridge / public API). Your solutions engineer notifies you in advance.

Pinning `1.0.1` is intentional: your App Store binary stays reproducible until you choose to upgrade the native SDK.

When Zelig publishes a new native version (e.g. `1.0.1` → `1.0.1`)

Zelig will:

1. Upload `ZeligStylerSDK-1.0.1.xcframework.zip` to GCS (`ios-widget-sdk-zelig-prod`).

2. Tag <https://github.com/wearezelig/ZeligStylerSDK-binary> as **1.0.1** (Package.swift + podspec updated with the new URL + SHA-256).
3. Tell you the new version number.

Then upgrade with **your** install method only:

Your install	What to do
A — Direct XCFramework	Download the new versioned zip (../ZeligStylerSDK-1.0.1.xcframework.zip) → remove the old framework → drag in the new one → Embed & Sign → rebuild → ship
B — CocoaPods	Bump the tag in the Podfile URL (../ZeligStylerSDK-binary/1.0.1/ZeligStylerSDK.podspec) → <code>rm -rf Pods Podfile.lock</code> && <code>pod install</code> → confirm Installing ZeligStylerSDK (1.0.1) → rebuild → ship
C — SPM	• Up to Next Major rule: File → Packages → Update to Latest Package Versions → confirm the resolved version → rebuild → ship. • Exact Version rule: change it to 1.0.1 first, then resolve → rebuild → ship.

Previous version zips remain online — old app builds keep resolving.

The SDK follows semantic versioning. Any **1.x** update is backward-compatible with your existing integration code. A future **2.x** would be a breaking change and would be called out explicitly (SPM's "Up to Next Major" would not auto-adopt it).

Step 11 — Release checklist

- ☐ Your **storeId** / **storeKey** are correct.
- ☐ **storeKey** not committed to source control.
- ☐ **productCode** passed and present in your store's catalog.
- ☐ Add-to-cart callback implemented; **completion** called on both success and failure.
- ☐ Logged-in **userId** passed for authenticated shoppers (**nil** only for guests).
- ☐ Tested on multiple device sizes (phone and tablet).

Troubleshooting

Symptom	Fix
Missing package product 'ZeligStylerSDK'	A previous install method wasn't fully removed. Check Package Dependencies, the Frameworks list, and the Podfile — keep exactly one.
Command PhaseScriptExecution failed (CocoaPods)	Xcode 15+ sandbox restriction. Build Settings → User Script Sandboxing → No (see Option B, step 4).

Symptom	Fix
Unable to resolve module 'ZeligStylerSDK'	Option B: you opened <code>.xcodproj</code> — open <code>.xcworkspace</code> instead. Option A: check target membership and the Embed & Sign setting.
Widget blank or not loading	Check network connectivity; verify <code>storeId</code> / <code>storeKey</code> ; open Safari Web Inspector and filter for <code>[Zelig iOS]</code> logs.
Add to cart hangs or never resolves	Confirm your <code>onAddToCart</code> / delegate method is wired up and that <code>completion(true/false)</code> is called on every code path.
Wrong product opens	Verify <code>productCode</code> is correct and exists in your store's catalog.
Touches not registering on a physical device	Contact your Zelig solutions engineer — this is typically a host-app layout configuration issue.

API reference

```

public enum ZeligStylerEntry { case productDetail, buildALook,
myCloset }

public struct ZeligStylerConfig {
    public let storeId: String
    public let storeKey: String
    public let productCode: String
    public let userId: String?
    public let entry: ZeligStylerEntry

    public init(
        storeId: String,
        storeKey: String,
        productCode: String,
        userId: String? = nil,
        entry: ZeligStylerEntry = .productDetail
    )
}

public struct ZeligCartItem {
    public let sku: String
    public let size: String
    public let quantity: Int?
    public let fromZelig: Bool?
}

public enum ZeligStylerError: Error {
    case loaderLoadFailed(URL)
    case widgetInitTimeout
    case modalMountTimeout

```

```

        case webViewNavigationFailed(underlying: Error)
        case hostPageUnavailable
    }

    public protocol ZeligStylerDelegate: AnyObject {
        // Required:
        func zeligStyler(_ styler: ZeligStyler,
                        didRequestAddToCart item: ZeligCartItem,
                        completion: @escaping (Bool) -> Void)

        // Optional:
        func zeligStylerDidRequestClose(_ styler: ZeligStyler)
        func zeligStyler(_ styler: ZeligStyler, didFailWith error:
            ZeligStylerError)
    }

    public final class ZeligStyler {
        public init(config: ZeligStylerConfig, delegate:
            ZeligStylerDelegate? = nil)

        /** Whether the widget has finished warming (true after
            prepare() / first open). */
        public var isReady: Bool { get }

        /** Fires when warming completes (main thread). May fire again
            on a reload. */
        public var onReady: (() -> Void)?

        /** Warm the widget in the background ahead of time.
            Idempotent – a no-op if
            already warmed for the same identity. Call early for an
            instant first open. */
        public func prepare()

        /** Present and open on the given product/entry. Near-instant
            if already warmed. */
        public func open(productCode: String = "", entry:
            ZeligStylerEntry = .productDetail,
                        from presenter: UIViewController, animated:
            Bool = true)

        public func present(from presenter: UIViewController,
            animated: Bool = true)
        public func dismiss(animated: Bool = true)
    }

    public struct ZeligStylerView: UIViewControllerRepresentable {
        public init(
            config: ZeligStylerConfig,
            onAddToCart: @escaping (ZeligCartItem, @escaping (Bool) ->
            Void) -> Void,
            onClose: (() -> Void)? = nil,

```

```
        onError: ((ZeligStylerError) -> Void)? = nil  
    )  
}
```

Support

Contact your Zelig solutions engineer for store credentials, environment configuration, and any merchant-specific setup.

Version: 1.0.1 • **Last updated:** July 2026